

SPIRE/SPIFFE: Concepts, Configuration, and Infrastructure

SPIFFE and SPIRE — the Concepts

SPIFFE (Secure Production Identity Framework For Everyone) is a standard, not software. It defines:

- A URI format for workload identity: `spiffe://<trust-domain>/<path>`
- What a valid credential (SVID) looks like
- How an agent presents that credential to workloads (the Workload API)

SPIRE is the reference implementation of SPIFFE: a server that acts as a CA and a registration database, plus per-node agents that handle attestation and SVID issuance.

What Is an SVID, and Who Is Attesting to Whom?

An SVID is an **identity document**, not a safety certificate. The analogy is a passport: a passport doesn't say "this person is trustworthy" — it says "this person is who they claim to be, and a government vouches for that." What the receiving party does with that identity is their own business.

This distinction matters because attestation flows in multiple directions simultaneously.

The out-of-band provisioning channel Both steps 0 and 1 below depend on material written to each node by `provision-tpm-devid.sh`, running from the out-of-band admin host over direct SSH before SPIRE starts. This script writes two things to `/etc/spire/` on every node:

- The node's own **DevID key pair and certificate** (step 1) — a TPM-bound key pair generated inside the node's hardware TPM, plus a certificate signed by the cluster DevID CA
- The server's **bundle-signing public key** (step 0) — read from `ipc4`'s TPM (handle `0x81008006`) and written as a PEM file

This direct-SSH channel is the trust anchor for the entire bootstrap. It is independent of Kubernetes: SPIRE cannot influence it, and a compromise of the Kubernetes control plane cannot reach it. A key delivered this way cannot be substituted by anyone who controls only the cluster. Both steps 0 and 1 derive their security from the integrity of this provisioning step, in addition to the hardware guarantees of the TPMs themselves.

Here is the full chain as implemented on this cluster:

0. Agent trusts the server (TPM-signed bundle) Before the SPIRE agent starts, the `verify-bundle` init container fetches the trust bundle JWT from `spire-bundle-signing` — a ClusterIP Service that exposes an HTTP endpoint on the SPIRE server pod (a busybox httpd sidecar on port 8181, serving the file that the `signer-unix` sidecar writes after signing with the server's TPM key). The init container verifies the RS256 signature on that JWT against `/etc/spire/server-bundle-signing.pub` — the server's TPM public key written by the provisioning script — and only writes the bundle to disk if the signature is valid. The agent will not start if this check fails.

The server's TPM public key lives in a file rather than being fetched from the TPM at runtime for two reasons. First, TPMs are local hardware — the server's TPM is physically on `ipc4` and there is no network protocol for reading a remote TPM's public key; each TPM is only accessible to the machine it is soldered to. Second, fetching it through any Kubernetes channel (ConfigMap, Secret,

API) would defeat the purpose: a key that arrives via Kubernetes can be swapped by anyone who controls Kubernetes.

The agent's trust in the server is therefore rooted in two things held together: the server's hardware TPM (which guarantees the private key never leaves ipc4's chip) and the provisioner's ability to deliver the matching public key to each agent node with integrity. Both must hold. The trust root is not in Kubernetes infrastructure.

1. Server trusts the agent (TPM DevID credential activation) DevID (IEEE 802.1AR Secure Device Identity) is the standard that defines a hardware-bound device credential: a key pair generated inside the agent host's TPM by the local administrator during enrollment (not at manufacturing — it is specific to this cluster's identity scheme, not indelible), so the private key never leaves the chip, paired with a certificate signed by an authority that vouches for which device the key belongs to. The DevID key pair and certificate are written to the node by the provisioning script described above. The agent sends its DevID certificate (signed by the cluster DevID CA) and proves it holds the corresponding private key by signing a server-issued nonce. The server then performs a credential activation challenge using the attesting agent node's EK (Endorsement Key) public key. The EK is a key pair burned into that agent node's TPM at manufacturing time; the manufacturer signs an EK certificate and stores it in that TPM's non-volatile memory, making the EK the hardware root of trust for that specific physical chip. The credential activation challenge encrypts a secret to the agent node's EK public key such that only the agent node's TPM holding that EK can decrypt it, and only if the DevID key is also loaded in that same TPM at the same time. The agent's TPM decrypts and returns the secret. Both proofs together — key possession and TPM residency — satisfy the server that the agent is running on a real, enrolled node.

2. Agent attests the workload (SO_PEERCREDS + kubelet) When a workload connects to `agent.sock`, the agent identifies it entirely by observation — `getsockopt(SO_PEERCREDS) → /proc/<pid>/cgroup → kubelet API` — without the workload asserting anything about itself. See the Workload Attestation section for the full mechanism.

3. Server issues an SVID to the workload via the agent The agent presents the observed workload identity to the server. If it matches a registration entry, the server signs an SVID (X.509 or JWT — see SVIDs below) and returns it to the agent, which hands it to the waiting workload process.

4. Workload presents the SVID to a peer The workload presents the SVID to whatever service it is calling — as a TLS client certificate for mTLS, or as a Bearer token in an Authorization header. See SVIDs below for when each form is appropriate.

5. Peer verifies against the trust bundle The receiving service verifies the SVID's signature against the SPIRE CA trust bundle. No pre-shared secrets, no manually distributed certificates. If the signature checks out and the SPIFFE ID matches the access policy, the connection is accepted.

Steps 0–3 are handled entirely by SPIRE. Steps 4–5 are between the workload and whatever it is talking to; SPIRE's job is to get a cryptographically-grounded identity into the workload's hands without the workload needing to manage secrets itself.

What an SVID Actually Proves

When a workload successfully receives an SVID, the credential embodies a four-layer combined claim:

1. **The node is genuine and enrolled** — TPM credential activation proved, through a hardware challenge that requires physical possession of the TPM chip, that the agent is running on a real node that was enrolled by the cluster administrator. This is hardware proof; it cannot be forged in software.
2. **The agent has attested to the SPIRE server** — the server accepted the agent's TPM DevID credential and issued it a Node SVID. The agent is now an authorized member of the cluster's trust domain.
3. **The pod is running in a specific namespace under a specific service account** — the agent observed this directly via `getsockopt(SO_PEERCREDS) → /proc → kubelet API`. These are Kubernetes-enforced properties: the namespace is set by the API server at admission, the service account token is issued by the API server at scheduling time, and neither can be changed by the pod itself after it starts.
4. **A registration entry explicitly authorizes this combination** — the cluster operator declared in advance that a pod in this namespace with this service account should receive this specific SPIFFE ID. The agent matched the observed pod metadata against that policy before issuing anything.

No single layer alone is sufficient. The TPM proves hardware identity but not which workload. The Kubernetes selectors prove pod identity but only as strongly as Kubernetes policy. The registration entry ties the two together under explicit operator authorization. An SVID that reaches a workload means all four layers held simultaneously.

SVIDs — the Two Forms

SVID = SPIFFE Verifiable Identity Document. Two formats:

X.509 SVID

A standard X.509 certificate with one key addition: the SPIFFE ID is encoded in the certificate's **Subject Alternative Name (SAN)** as a URI (`spiffe://ipc.local/demo-app`). Inspect with:

```
openssl x509 -in svid.pem -text -noout | grep URI
```

Properties:

- **Short-lived** (default TTL: 1 hour). The agent rotates them automatically before expiry, typically at the 50% mark.
- Carries the full chain: leaf cert + intermediate(s) up to the trust bundle.
- Used for **mTLS** — both sides present their SVID as the TLS client/server cert, and verify the peer's cert against the trust bundle. This is what experiment 21 demonstrates.
- Opaque to network intermediaries (the identity is in the cert, not the payload).

JWT SVID

A signed JWT with the SPIFFE ID in the `sub` claim and an `aud` (audience) claim.

Properties:

- Also short-lived. **Not** automatically rotated — the workload must re-fetch when it needs a fresh one.
- Used when mTLS is not possible: HTTP APIs where identity must be passed in a header (`Authorization: Bearer <jwt>`), or when a network intermediary (load balancer, API gateway) terminates TLS before reaching the workload.
- The receiving service verifies the JWT signature against the trust bundle’s public key, not by doing a TLS handshake.
- Fetch with: `spire-agent api fetch jwt -audience <target-service> -socketPath ...`
- The **audience field matters**: a JWT fetched for audience `api-gateway` won’t be accepted by a verifier expecting `payments-service`. This prevents token replay across services.

When to Use Which

Scenario	Format
Service-to-service where you control both ends	X.509 (mTLS)
HTTP API, identity passed in header	JWT
Identity through a TLS-terminating proxy	JWT
Authenticating to Vault (JWT auth method)	JWT
Envoy/service mesh transparent mTLS	X.509

Why this cluster uses X.509: The experiments here (`mtls-demo`, `envoy-demo`) use mTLS — both sides of each connection are SPIRE-attested Kubernetes workloads that can participate in a full TLS handshake. X.509 is the right choice: identity travels in the certificate, rotation is automatic, and no application-level token handling is needed.

When you would use JWT instead (or in addition): JWT becomes necessary when one end of the connection cannot participate in mTLS — either because it doesn’t support client certificate authentication, or because TLS is terminated by an intermediary before reaching the workload. The connection is still typically HTTPS: the server presents its certificate and the client validates it, so the channel is encrypted and the client knows it is talking to the right server. But only one side is authenticated at the TLS layer. The JWT Bearer token in the `Authorization` header fills the other half — it proves the caller’s identity at the application layer rather than the TLS layer.

Common cases:

- **Vault authentication** — Vault’s JWT auth method expects a Bearer token, not a client cert. A workload that needs to fetch secrets from Vault calls the Workload API to get a JWT SVID with audience `vault`, presents it to Vault, and receives a Vault token in return.
- **REST APIs behind an API gateway** — the gateway terminates TLS; the workload’s X.509 identity never reaches the backend. The workload instead fetches a JWT SVID and sends it as an `Authorization: Bearer` header that the gateway or backend can validate.
- **External or third-party services** — services outside the cluster that accept JWTs but have no SPIFFE awareness.

How the called service validates a Bearer token: The verification is structurally identical to X.509 chain validation — the same trust anchor, applied to a different encoding. In the X.509 case, the server checks that the client certificate is signed by a CA in the trust bundle and reads the SPIFFE

ID from the SAN URI. In the JWT case, the server splits the token into header/payload/signature, verifies the RS256 or ES256 signature against the SPIRE trust bundle's public key, and reads the SPIFFE ID from the `sub` claim. It also checks `aud` (must match this service), `exp` (must not be expired), and `iss` (must be the expected SPIRE server). Both reduce to: "does a signature made by the SPIRE CA's private key cover this credential, and does the credential assert the identity I expect?"

The called service has three options for how it performs this validation:

1. **Application code with a JWT library** — the service loads the trust bundle's public key and calls something like `jwt.verify(token, publicKey, { audience: "my-service" })`. Explicit but requires the application to know about SPIRE.
2. **API gateway or sidecar validates on behalf of the service** — the gateway extracts the Bearer token, validates it against the SPIRE JWKS endpoint, and either rejects the request or passes a verified identity header (e.g. `X-SPIFFE-ID: spiffe://ipc.local/caller`) to the backend. The backend never touches the JWT itself.
3. **SPIFFE SDK** — the SDK watches the Workload API for trust bundle updates and validates incoming JWTs automatically, keeping the validation current as the bundle rotates.

Using both simultaneously: A workload at the boundary between these two worlds can hold both at once. For example: it participates in mTLS with internal peers (X.509, rotated automatically by the agent) while also authenticating to Vault and calling external REST APIs (JWT, re-fetched by the application when needed). The Workload API issues whichever forms are requested; there is no restriction on fetching both. The same SPIRE trust bundle verifies both forms — its CA cert validates X.509 chains, its public key verifies JWT signatures.

The Trust Bundle

The trust bundle is the SPIRE CA's root certificate(s). Everything signed by SPIRE is verifiable against it. On this cluster:

- The server writes it to the `spire-bundle` ConfigMap in the `spire` namespace (via the `k8s_bundle` notifier) as a reference copy, but agents no longer bootstrap from it directly — they use the TPM-verified bundle written by the `verify-bundle` init container.
- Workloads receive it alongside their SVID when calling the Workload API — so they can verify peers without any pre-shared secrets or manual cert distribution.

Trust bundle rotation: If the server's CA rotates (either on schedule or forced), it publishes the new root alongside the old one during a transition window. SVIDs issued under the old CA remain valid until expiry; verifiers must accept both roots during the overlap. This is automatic in SPIRE.

What Is Running on This Cluster

Trust domain: `ipc.local`

SPIRE Server — StatefulSet (1 replica) on the cluster. Two things persist in SQLite on a 1Gi PVC provisioned by the `nfs-subdir-external-provisioner` StorageClass, backed by `nazgul`

(192.168.89.2) at `/mnt/primary_storage/k8s-nfs:`

- **CA keys** — the private key material the server uses to sign SVIDs (Secure Verifiable Identity Documents), the short-lived certificates issued to workloads as proof of identity.
- **Registration entries** — the policy database that maps Kubernetes pod attributes (namespace, service account) to SPIFFE IDs. When a workload connects to the Workload API, the agent matches the pod’s observed metadata against these entries to determine which identity to issue. See the Registration Entries section for a full explanation.

Listens on gRPC port 8081 via a headless service. Flux manages it from `manifests/spire/`.

SPIRE Agent — DaemonSet on all 6 nodes. Runs with `hostPID: true` (required for workload attestation — see Workload Attestation). Exposes the Workload API at `/run/spire/sockets/agent.sock` on the node’s host filesystem via a `hostPath` volume, so any pod on the same node that mounts that path can reach it. Bootstraps its trust bundle via a `verify-bundle` init container that fetches the server’s TPM-signed bundle before the agent starts (see TPM Server Attestation). Attests to the SPIRE server via TPM DevID (see Node Attestation).

Cluster Architecture

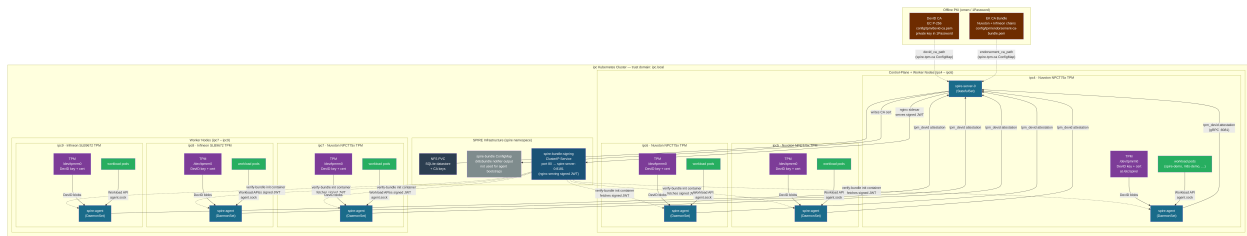


Figure 1: SPIRE cluster architecture — nodes, agents, server, TPMs, and workloads

Node Attestation

The agent can’t just claim “I’m a legitimate node” — the server needs proof. **Node attestation** is how the agent proves to the server that it’s running on a real, authorized Kubernetes node. This cluster uses **TPM DevID** attestation, documented in the sections below.

k8s_psat (Kubernetes Projected Service Account Token) — background only

This cluster originally used `k8s_psat` before being upgraded to TPM DevID. The description below is retained as conceptual background; it is not the active mechanism. The `k8s_psat` registration entry still exists in the server database but is unused.

The `k8s_psat` flow:

1. Kubernetes issues a short-lived projected SAT to the agent pod (audience: `spire-server`, expiry: 7200s). This token is in the pod’s filesystem at `/var/run/secrets/tokens/`.
2. The agent sends this token to the SPIRE server over gRPC.
3. The server calls the Kubernetes **TokenReview API** to validate it — Kubernetes confirms the token is real, which SA issued it, and which pod it was issued to.

4. The server checks its allow-list (`spire:spire-agent SA`) and grants the agent a **Node SVID** with SPIFFE ID `spiffe://ipc.local/k8s-node`.

From that point on, the agent authenticates to the server using the Node SVID (mTLS on the gRPC channel). The PSAT is only used once for initial bootstrapping. The trust root is “Kubernetes itself validated this token” — which means a compromised Kubernetes control plane breaks node attestation. TPM DevID removes this dependency.

TPM DevID Node Attestation Flow

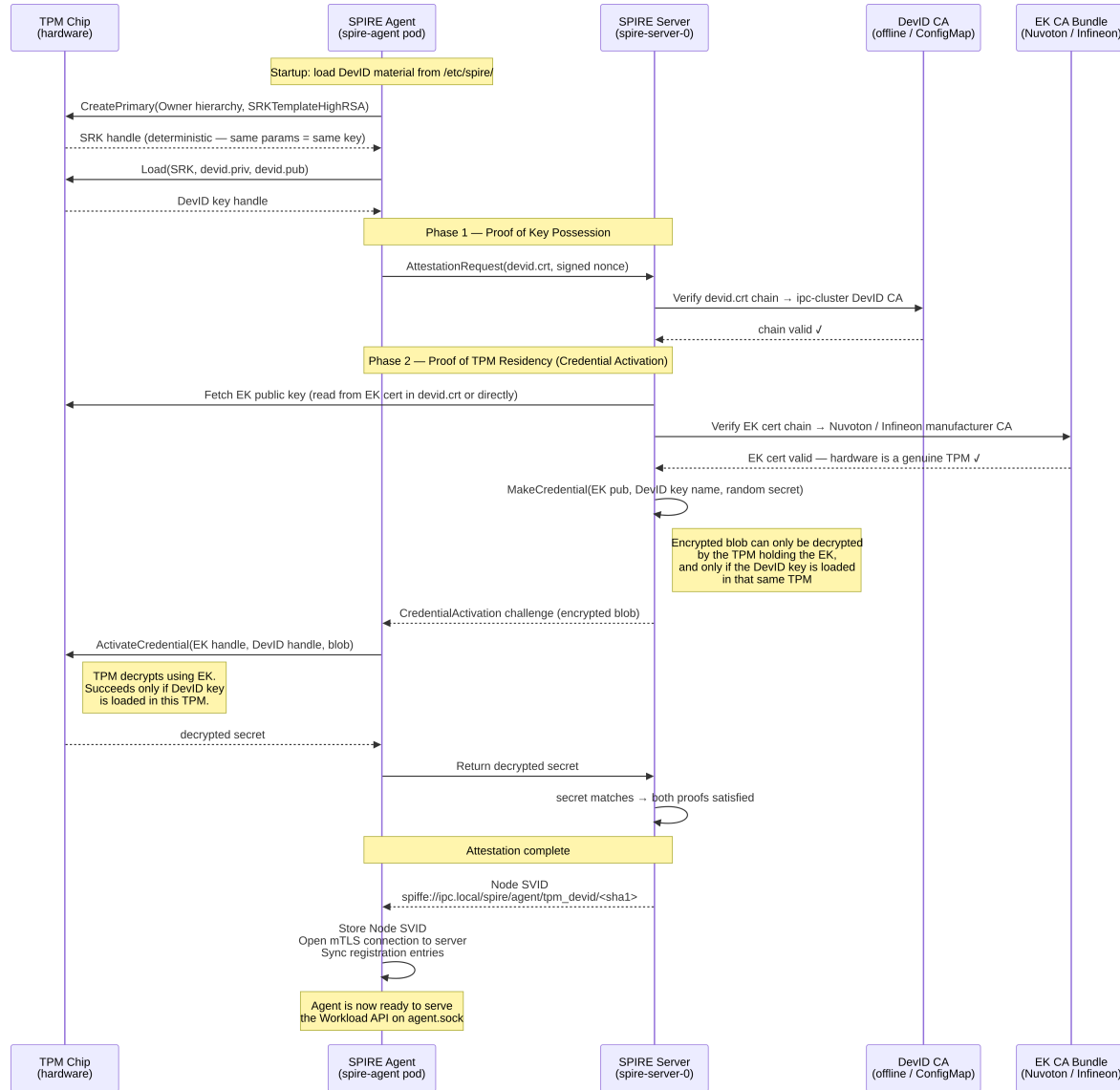


Figure 2: TPM DevID node attestation sequence

Workload Attestation

After a node is attested, the agent needs to verify that a calling workload is what it claims to be before issuing an SVID. This happens entirely on-node; no server round-trip is needed. Critically, **the workload never gets to assert its own identity** — the agent determines it entirely by observation from outside the workload process.

The flow for this cluster (k8s workload attestor):

1. Workload connects to the agent socket. The workload process opens a connection to `/run/spire/sockets/agent.sock`, a Unix domain socket on the node filesystem, mounted into the workload pod via a `hostPath` volume.

2. Agent calls `getsockopt(SO_PEERCREC)` → host PID. Unix domain sockets have a kernel feature called `SO_PEERCREC`. When the agent calls `getsockopt(SO_PEERCREC)` on the connected socket, the kernel returns the PID, UID, and GID of the process on the other end. This is trustworthy because it comes from the kernel — the connecting process cannot forge it. The result is the *host* PID (e.g. 48721), which is why `hostPID: true` is required on the agent `DaemonSet`: without it, PIDs are namespaced inside the pod and `getsockopt(SO_PEERCREC)` returns a PID that is meaningful only within that pod’s PID namespace and cannot be looked up in the host’s `/proc`.

`hostPID: true` is a requirement of this specific implementation of the k8s attestor — it uses host PIDs to walk `/proc` and identify containers. It is not a fundamental requirement of the observation-based approach in general. A different implementation that could map a Unix socket peer to a container ID through a namespace-aware kernel interface would not need `hostPID: true` and would still require no modification to the workload. The workload is unaware of SPIRE in either case — it simply opens a socket connection.

3. Agent reads `/proc/<pid>/cgroup` → container ID. The cgroup path for a containerized process encodes the container ID, in a form like:

```
0::/kubepods/besteffort/pod<pod-uid>/<container-id>
```

The agent parses this to extract the container ID. (`use_new_container_locator: true` in the agent config tries `mountinfo` first, which is more reliable under `cgroupv2`, before falling back to the cgroup path.)

4. Agent calls kubelet API → pod metadata. The agent calls the local kubelet’s `/pods` endpoint (this is why the agent needs `nodes/proxy` RBAC) asking: “which pod contains a container with this ID?” The kubelet returns the full pod record: namespace, pod name, service account, labels, and annotations. The agent now has everything Kubernetes knows about who made the connection.

5. Agent matches pod metadata against registration entries. The agent compares the pod’s namespace and service account against its registration entries (syncd from the SPIRE server). If an entry matches — for example, `ns=mtls-demo, sa=mtls-client-sa → spiffe://ipc.local/mtls-client` — the agent issues an SVID for that SPIFFE ID.

The chain is: *which process connected* (kernel) → *which container holds that process* (procs) → *which pod owns that container* (kubelet) → *which SPIFFE ID applies* (registration entries). Each step is derived from a source the workload cannot influence. The workload requires no modification — no SDK, no sidecar, no injected token — because identity is determined entirely by observation

from outside the workload process. This property holds for any observation-based attestor, not just this specific /proc-based implementation.

Workload Attestation Flow

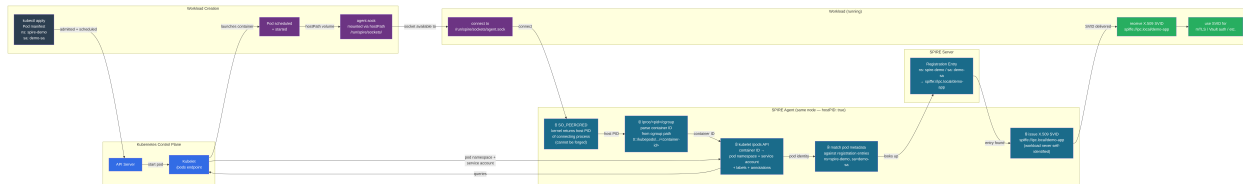


Figure 3: Workload attestation flow

Pelagos Compatibility Fixes

Two Pelagos bugs affected SPIRE workload attestation on this cluster:

Bug	Symptom	Fixed
hostPID: true ignored (#299)	SO_PEERCRECRED returned PID 0; server rejected	v0.65.7
32-char container IDs (#301)	SPIRE regex expected 64-char hex; attestation failed	v0.65.8

Both are fixed in the running version (v0.65.47).

Workload Configuration

A workload does not automatically get access to the Workload API socket — the pod spec must explicitly declare it, and the application must know to connect to it.

Pod spec requirements:

```
volumes:
- name: spire-agent-socket
  hostPath:
    path: /run/spire/sockets
    type: Directory
containers:
- name: my-app
  volumeMounts:
- name: spire-agent-socket
  mountPath: /run/spire/sockets
  readOnly: true
```

The application then connects to `/run/spire/sockets/agent.sock`. This path is a convention shared across the SPIFFE ecosystem — the SPIFFE Workload API SDK defaults to it, and `spire-agent api fetch` uses it. It can be overridden via an environment variable or flag if needed.

What “no workload modification” actually means in practice:

The workload *process* does not need to be rebuilt or instrumented — it just opens a socket. But the *pod spec* does need to be modified to mount the socket directory, and the application needs to know the socket path and how to call the Workload API (either via the SDK, via a helper like spiffe-helper that writes cert files to disk, or via Envoy SDS if you are using a service mesh). Migrating an existing application that uses manually-managed TLS certificates to SPIRE means changing the pod spec and changing how the application loads its credentials — the binary itself may be unchanged, but the deployment is not zero-effort.

Automatic injection via mutating admission webhook:

The pod spec modification can be eliminated entirely with a **mutating admission webhook** — a Kubernetes controller that intercepts pod creation and automatically injects the socket volume mount into any pod carrying a particular label or annotation. SPIRE does not ship one by default, but the pattern is well-established. The SPIRE Controller Manager and spiffe-helper projects both support variations of this. This cluster does not currently run a webhook, so socket mounts must be added to pod specs manually.

Service account:

Every pod runs under a Kubernetes service account, declared in the pod spec via `serviceAccountName`. If not specified, Kubernetes assigns the `default` service account for the namespace. The service account must exist as a `ServiceAccount` resource in the same namespace — Kubernetes then automatically mounts a signed JWT for it into the pod at startup. The service account is how Kubernetes RBAC controls what the pod can do with the Kubernetes API.

For SPIRE, the service account is also an identity anchor. When the agent resolves a connecting process to a pod via the kubelet API, one of the fields returned is `serviceAccountName`. SPIRE uses this — alongside the namespace — as the selector that determines which registration entry applies and therefore which SPIFFE ID to issue.

Registration entry:

The pod spec change gives the workload access to the socket, but the SPIRE server also needs a registration entry that maps the pod's namespace and service account to a SPIFFE ID. Three things must agree for a workload to receive an SVID:

```
ServiceAccount resource:  name: mtls-client-sa    (namespace: mtls-demo)
                           ^
Pod spec:                  serviceAccountName: mtls-client-sa
                           ^
SPIRE entry:               k8s:ns:mtls-demo + k8s:sa:mtls-client-sa → spiffe://ipc.local/mtls-
client
```

If any one of these disagrees — wrong namespace, wrong service account name, entry not yet created — the workload gets `PermissionDenied: no identity issued`. The agent gives no specific indication of which piece is missing; it simply sees a pod whose metadata matches no entry.

The service account and pod spec live in Git and are managed by Flux. The registration entry currently does not — it lives in SQLite on the NFS PVC, created imperatively via `spire-server entry create`. This is the GitOps gap: if the cluster is rebuilt, Flux restores the pod and service account automatically, but the SPIRE entry is gone and the workload silently fails to get an SVID. See Registration Entries below for the full discussion and the `ClusterSPIFFEID` alternative that closes this gap.

Registration Entries

A registration entry is the answer to the question: “which workloads on this cluster should receive a SPIFFE identity, and what identity should they get?” Without a registration entry, a workload can connect to the Workload API socket and the agent can fully identify it — but then has nothing to match it against, so it returns `PermissionDenied: no identity issued`. The entry is the policy that bridges observation (what the agent sees) to identity (what SVID to issue).

Each entry has three fields:

SPIFFE ID — the identity to issue if this entry matches. For example, `spiffe://ipc.local/mtls-client`. This becomes the Subject Alternative Name in an X.509 SVID or the `sub` claim in a JWT SVID.

Parent ID — who must have already attested the caller before this entry applies. For workloads, this is always a node alias (`spiffe://ipc.local/k8s-node`), meaning “the workload must be running on a node whose agent has already attested to the server.” This creates a two-level hierarchy: nodes attest first, then workloads on those nodes can receive identities. You could not issue an SVID to a workload on a rogue node that never attested, because the parent requirement would not be satisfied.

Selectors — the conditions that must all match for the entry to apply. For the `k8s` workload attester these are Kubernetes-native attributes the agent observed: - `k8s:ns:<namespace>` — the pod’s namespace - `k8s:sa:<service-account>` — the pod’s service account - `k8s:pod-label:<key>:<value>` — a pod label (optional, for finer-grained policy)

All selectors in an entry must match simultaneously. An entry with `k8s:ns:mtls-demo` and `k8s:sa:mtls-client-sa` only matches a pod that is in the `mtls-demo` namespace AND uses the `mtls-client-sa` service account. A pod in `mtls-demo` using a different service account gets nothing.

Node alias entries — there is a special class of entry where the SPIFFE ID represents a node rather than a workload. On this cluster:

```
SPIFFE ID:  spiffe://ipc.local/k8s-node
Parent:     spiffe://ipc.local/spire/server
Selector:   tpm_devid:issuer:cn:ipc-cluster DevID CA
```

This entry says: any agent that attests via `tpm_devid` with a cert signed by our DevID CA receives the node alias `spiffe://ipc.local/k8s-node`. Workload entries then use that alias as their Parent ID. This indirection means you don’t need a separate workload entry per node — any workload on any attested node can match, because they all share the same node alias as their parent.

What the agent does with entries — the server syncs the full set of registration entries to every agent. When a workload connects, the agent evaluates all entries whose Parent ID matches the node’s own SPIFFE ID, checks the selectors against the observed pod metadata, and issues an SVID for each entry that matches. One workload pod can match multiple entries and receive multiple SVIDs.

How to Create and View Entries

Entries are managed via the SPIRE server CLI:

```
# View all entries
kubectl exec -n spire spire-server-0 -c spire-server -- \
  /opt/spire/bin/spire-server entry show

# Create a workload entry
kubectl exec -n spire spire-server-0 -c spire-server -- \
  /opt/spire/bin/spire-server entry create \
  -spiffeID spiffe://ipc.local/my-app \
  -parentID spiffe://ipc.local/k8s-node \
  -selector k8s:ns:my-namespace \
  -selector k8s:sa:my-service-account
```

Current Entries on This Cluster

SPIFFE ID	Parent	Selectors	Notes
spiffe://ipc.local/k8s-node	spire/server	tpm_devid:issuer:cn:ipc-cluster DevID CA	Node alias (active)
spiffe://ipc.local/k8s-node	spire/server	k8s_psat:cluster:ipc	Node alias (legacy, unused)
spiffe://ipc.local/demo-app	k8s-node	k8s:ns:spire-demo, k8s:sa:demo-sa	Experiment 11
spiffe://ipc.local/mtls-server	k8s-node	k8s:ns:mtls-demo, k8s:sa:mtls-server-sa	Experiment 21
spiffe://ipc.local/mtls-client	k8s-node	k8s:ns:mtls-demo, k8s:sa:mtls-client-sa	Experiment 21
spiffe://ipc.local/envoy-server	k8s-node	k8s:ns:envoy-demo, k8s:sa:envoy-server-sa	Experiment 24
spiffe://ipc.local/envoy-client	k8s-node	k8s:ns:envoy-demo, k8s:sa:envoy-client-sa	Experiment 24
spiffe://ipc.local/sidecar-server	k8s-node	k8s:ns:sidecar-demo, k8s:sa:sidecar-server-sa	Experiment 24
spiffe://ipc.local/sidecar-client	k8s-node	k8s:ns:sidecar-demo, k8s:sa:sidecar-client-sa	Experiment 24

GitOps Gap

Entries live in SQLite on the NFS PVC, not in Git. If the PVC is lost or the cluster is rebuilt, all entries must be re-created manually. The current workaround is to keep the `spire-server entry create` commands in the experiment manifests as Jobs so they can be re-run.

GitOps Alternative: ClusterSPIFFEID

The **SPIRE Controller Manager** provides a `ClusterSPIFFEID` CRD. Entries are declared as Kubernetes resources in Git; the controller reconciles them into SPIRE continuously. This closes the GitOps gap. Architected in `docs/spire-trust-chain-clusterspiffeid.mmd` but not yet deployed.

TPM Hardware on This Cluster

TPM Keys in Use

Three distinct TPM keys are involved in this cluster’s attestation scheme. It is easy to confuse them because they are all TPM-bound RSA keys, but they have different origins, purposes, and scopes:

Key	Lives on	Created by	Purpose	Files / Handle
Endorsement Key (EK)	Every node’s TPM	Manufacturer, at the factory	Proves a TPM is genuine hardware; used only during credential activation challenge	EK cert in TPM NV; read with <code>tpm2_getekcertificate</code>
DevID key	Every node’s TPM	<code>provision-tpm-devid.sh</code> , at enrollment	Proves which enrolled node an agent is running on; used for every attestation	<code>/etc/spire/devid.priv</code> (blob), <code>/etc/spire/devid.pub</code> (blob), <code>/etc/spire/devid.crt</code> (cert)
Bundle-signing key	ipc4’s TPM only	<code>provision-server-bundle-signing.sh</code> , once	Signs the SPIRE trust bundle JWT so agents can verify the server before trusting it	Handle 0x81008006 on ipc4; public key distributed to all nodes as <code>/etc/spire/server-bundle-signing.pub</code>

EK — indelible, manufacturer-provisioned, one per chip, proves hardware authenticity. Cannot be changed or regenerated by the administrator.

DevID key — administrator-provisioned at enrollment, one per node, proves cluster identity. Regenerated whenever a node is reinstalled (`provision-tpm-devid.sh` runs automatically via `reinstall-nodes.sh`). The private key blob is TPM-bound: it can only be loaded and used inside the specific TPM that created it.

Bundle-signing key — provisioned once on ipc4 (the SPIRE server node), never regenerated unless ipc4 is reinstalled. Its public key is the out-of-band trust anchor that agents use to verify the server independently of Kubernetes. It must be re-distributed to all agent nodes if the key is ever regenerated.

All six nodes have TPM 2.0 chips with manufacturer-provisioned Endorsement Key (EK) certificates stored in TPM NV (non-volatile) memory. The EK cert is the hardware root of trust: it is signed by the manufacturer’s CA at the factory and proves that the EK private key is bound to a specific, real TPM chip. The EK private key never leaves the chip.

EK Certificate Inventory

Node	Chip	Manufacturer	EK Cert Issuer	Valid Until
ipc4	Nuvoton NPCT75x	Nuvoton (TW)	NPCTxxx ECC384 LeafCA 022111	2043
ipc5	Nuvoton NPCT75x	Nuvoton (TW)	NPCTxxx ECC384 LeafCA 022111	2043
ipc6	Nuvoton NPCT75x	Nuvoton (TW)	NuvotonTPM- RootCA2210	2042
ipc7	Nuvoton NPCT75x	Nuvoton (TW)	NuvotonTPM- RootCA2211	2042
ipc8	Infineon SLB9672	Infineon (DE)	OPTIGA(TM) TPM 2.0 RSA CA 061	2037
ipc9	Infineon SLB9672	Infineon (DE)	OPTIGA(TM) TPM 2.0 RSA CA 061	2037

`tpm2-tools` is installed on all nodes (`apt install tpm2-tools`). The EK cert can be read with `sudo tpm2_getekcertificate -o /tmp/ek.crt`.

Are Nuvoton and Infineon Equivalent?

For SPIRE attestation: yes, completely. The TPM 2.0 attestation protocol is standardized by the TCG (Trusted Computing Group) and is identical regardless of manufacturer. SPIRE validates the EK cert chain and does a challenge-response over the EK private key; the manufacturer is irrelevant to the protocol.

In general terms, Infineon OPTIGA is the dominant TPM manufacturer globally and is used in FIPS-certified and government-cleared systems. Nuvoton NPCT is a legitimate TCG-certified TPM 2.0 part commonly shipped by HP in their business machines. Neither has active unpatched vulnerabilities. The historical ROCA vulnerability (CVE-2017-15361) affected Infineon RSA key *generation* in old firmware; ipc8/9 run firmware 15.0.0.22, well past the patched version.

ipc8/9 have both RSA and ECC EK certs (Infineon provisions both; Nuvoton nodes only have one). ECC P-384 is more modern than RSA 2048 — a mild theoretical advantage, not meaningful in practice.

ipc6 and ipc7 have different Nuvoton root CAs (2210 vs 2211) despite being the same hardware model. This reflects different manufacturing batches. For SPIRE attestor configuration you will need to trust all four manufacturer CA chains: two Nuvoton, one Infineon RSA, one Infineon ECC.

What Happens After 2037 (ipc8/9 EK Cert Expiry)?

You do not replace the TPM or the hardware. The TPM chip is permanent — it's soldered to the motherboard. What expires in 2037 is the manufacturer-signed *certificate* for the EK public key, not the EK key itself. The hardware continues to function normally.

What changes is your attestation policy. Currently, full EK cert chain validation works like this: “I trust this node because its EK cert chains to Infineon’s CA, which means Infineon vouches that this is a real TPM.” After expiry, that chain is stale.

The alternative — which is always available and is what most production deployments use alongside cert-chain validation — is **EK public key fingerprinting**: during initial enrollment, you record the EK public key of each node. The attester then checks: “I trust this node because it can prove possession of the private key corresponding to *this enrolled public key*.” There is no expiry on a public key fingerprint. You lose the manufacturer-provenance guarantee (that the key is in a real TPM) but retain the cryptographic identity guarantee (that this is the same hardware you enrolled).

In practice for this cluster: re-enrollment in 2037 is the right move — you run through node attestation setup once more at that point and record the EK fingerprints. Or you pre-enroll fingerprints now alongside cert chain validation, and the transition in 2037 is a config change rather than a re-enrollment.

TPM Attestation: Plugin Choice and Implementation Plan

Plugin: `tpm_devid` (built-in to SPIRE)

SPIRE 1.9.6 has a native TPM node attester called `tpm_devid`, stable since SPIRE 1.0.1 (2021). No external plugins are needed. The Bloomberg community plugin (`bloomberg/spire-tpm-plugin`) is archived and unmaintained — ignore it.

How `tpm_devid` Attestation Works

The attestation is two-part, which is what makes it stronger than a simple hardware value check:

1. **Proof of key possession** — the agent signs a server-issued nonce with its DevID private key. The server verifies the signature against the DevID certificate’s public key.
2. **Proof of TPM residency** — the server performs a **credential activation challenge** using the node’s EK public key. It encrypts a secret to the EK such that only the TPM holding the EK can decrypt it, and only if the DevID key is also loaded in that same TPM. The agent returns the decrypted secret, proving both keys reside in the same physical chip.

This two-part proof means an attacker can’t just copy key material off a node — possession of the DevID private key blob is not enough; it must be activated inside the TPM that holds the EK.

The SPIFFE ID issued by the `tpm_devid` attester takes the form: `spiffe://ipc.local/spire/agent/tpm_devid/<fingerprint-of-DevID-cert>`

What `tpm_devid` Requires

Per node (out-of-band provisioning): - A **DevID key pair generated inside the TPM** — the private key never leaves the chip; it exists only as an encrypted TPM key blob (`devid.priv`) paired with a public key blob (`devid.pub`) - A **DevID certificate** signed by a CA you control (the DevID CA), containing the public key

SPIRE Server: - `devid_ca_path` — the DevID CA cert (validates the DevID cert chain on each agent) - `endorsement_ca_path` — all manufacturer EK CA certs (validates the EK cert chain, completing the hardware root-of-trust verification)

This cluster requires four EK CA certs: Nuvoton NPCTxxx LeafCA 022111, NuvotonTPM-RootCA2210, NuvotonTPMRootCA2211, and Infineon OPTIGA TPM 2.0 RSA CA 061.

SPIRE Agent: - `devid_cert_path` — path to the signed DevID certificate (PEM) - `devid_priv_path` — path to the TPM private key blob - `devid_pub_path` — path to the TPM public key blob

Migration Impact

Switching from `k8s_psat` to `tpm_devid` invalidates all existing agent registrations — the Node SVID format changes. The SPIRE server's registration database must be cleared and the registration jobs re-run after the switch. Since entries are currently managed by imperative Jobs, this is re-running those jobs, not a data migration.

TPM Attestation: Implementation — What Was Built

All seven phases are complete. This section documents what was actually implemented, including the non-obvious technical details discovered during the process.

Phase 1 — DevID CA (complete)

An offline EC P-256 CA (20-year validity) was generated on the out-of-band admin host and used exclusively to sign node DevID certificates.

- **Private key:** stored in 1Password as `ipc-cluster DevID CA key` (Private vault, notes field). Never written to disk longer than the signing operation; shredded immediately after storage.
- **Public cert:** `config/tpm/devid-ca.pem` — committed to Git (public material only).
- **Signing:** the provisioning script retrieves the key via `op item get ... | tr -d '"'` (the `op` CLI wraps field values in double quotes; stripping is required for the PEM to be valid).

Phase 2 — Manufacturer EK CA Bundle (complete)

Six CA certificates assembled into `config/tpm/endorsement-ca-bundle.pem`. All URLs were discovered from the AIA (Authority Information Access) extensions in the live EK certificates on each node via `tpm2_getekcertificate + openssl x509 -text`.

The bundle is two full chains, not four flat certs:

Chain	Nodes	Certs
Nuvoton ECC521	ipc4, ipc5	Root (NPCTxxx ECC521 RootCA) → Intermediate (NPCTxxx ECC384 LeafCA 022111)
Nuvoton 2210	ipc6	Self-signed root (NuvotonTPMRootCA2210)
Nuvoton 2211	ipc7	Self-signed root (NuvotonTPMRootCA2211)
Infineon RSA	ipc8, ipc9	Root (OPTIGA RSA Root CA 2) → Intermediate (OPTIGA TPM 2.0 RSA CA 061)

The EK CA bundle is the proof that an EK cert was signed by a real manufacturer. Without it, SPIRE cannot verify that the TPM presenting the EK is a genuine hardware chip.

Phase 3 — Node Provisioning Script (complete)

`scripts/provision-tpm-devid.sh` runs from the out-of-band admin host, provisions one node end-to-end, and is idempotent (skips if a valid cert signed by our CA already exists with >30 days to expiry).

What the script does:

1. Installs `tpm2-openssl` on the node (`apt install tpm2-openssl`)
2. Creates the SRK (Storage Root Key) primary under the **Owner hierarchy** (`-C o`)
3. Creates the DevID child key under the SRK
4. Loads the key and persists it to a temporary NV handle (`0x81010001`)
5. Generates a CSR via the `tpm2 OpenSSL` provider — the TPM signs the CSR internally
6. Evicts the NV handle — SPIRE uses blob files, not handles
7. Strips the 2-byte TPM2B wrapper from both blob files (see below)
8. Scps the CSR to the out-of-band admin host; retrieves the DevID CA key from 1Password; signs the cert
9. Installs the signed cert at `/etc/spire/devid.crt`; verifies it chains to the DevID CA
10. Cleans up temporary files; leaves `/etc/spire/` with `devid.crt`, `devid.priv`, `devid.pub`, `devid_pubkey.pem`

Critical implementation details discovered during integration:

Primary key hierarchy: The primary must be under the **Owner hierarchy** (`-C o`), not the Endorsement hierarchy (`-C e`). SPIRE's `tpm_devid` plugin recreates the primary at startup using `SRKTemplateHighRSA()` from `go-tpm-tools`, which calls `tpm2.CreatePrimaryEx` against `tpm2.HandleOwner`. In the TPM, primary keys are deterministic — same parameters + same hierarchy = same key. If the provisioning script uses the Endorsement hierarchy but SPIRE uses the Owner hierarchy, the primaries are different keys and SPIRE cannot load the DevID child blobs.

Primary key attributes (must match SPIRE's `SRKTemplateHighRSA()` exactly):

```
-G rsa
-a "fixedtpm|fixedparent|sensitivedataorigin|userwithauth|noda|restricted|decrypt"
```

The symmetric cipher (AES-128-CFB) is implied by `restricted|decrypt` in `tpm2-tools` 5.7. Do NOT use `-G rsa2048:aes128cfb:null` — that compound form is not accepted.

DevID key signing scheme must be `rsa:rsassa` (RSASSA-PKCS1-v1.5), not `rsa` (which defaults to `null` scheme). With a `null` scheme, `go-tpm`'s `DecodePublic` returns a `Public` struct where `RSAParameters.Sign` is `nil`, causing a `nil` pointer dereference in the `tpm_devid` plugin.

Blob format — stripping the TPM2B prefix: `tpm2_create -u/-r` outputs blobs in `TPM2B_PUBLIC` / `TPM2B_PRIVATE` format (2-byte big-endian size + payload). SPIRE's `go-tpm` library expects the raw payload without that prefix: `- tpm2.DecodePublic(pubBytes)` decodes raw `TPMT_PUBLIC` directly - `tpm2.Load(... privateBlob)` packs the private blob via `U16Bytes()`, adding the prefix itself — so the file must NOT already have one

The strip must happen **after** `tpm2_load` (which needs the full TPM2B wrappers to load the key), not before:

```
dd if=devid.pub bs=1 skip=2 of=devid.pub.raw && mv devid.pub.raw devid.pub
dd if=devid.priv bs=1 skip=2 of=devid.priv.raw && mv devid.priv.raw devid.priv
```

CSR generation via tpm2-openssl: Standard openssl req -key cannot use a TPM public key PEM as if it were a private key. Instead: install tpm2-openssl, persist the DevID key to a TPM NV handle, and generate the CSR through the TPM provider:

```
openssl req -provider tpm2 -provider default -key "handle:0x81010001" -new -out devid.csr ...
```

The private key signs the CSR data inside the TPM chip. After the CSR is fetched, evict the handle — SPIRE only needs the blob files.

Phase 4 — SPIRE Server Config (complete)

manifests/spire/server-config.yaml: k8s_psat attestor replaced with tpm_devid:

```
NodeAttestor "tpm_devid" {
  plugin_data {
    devid_ca_path      = "/run/spire/tpm-ca/devid-ca.pem"
    endorsement_ca_path = "/run/spire/tpm-ca/endorsement-ca-bundle.pem"
  }
}
```

manifests/spire/server-tpm-ca.yaml: new ConfigMap carrying both CA files, mounted into the StatefulSet at /run/spire/tpm-ca/.

manifests/spire/server-rbac.yaml: the server ClusterRole was stripped of tokenreviews, subjectaccessreviews, nodes, and pods. With tpm_devid, the server does not call the Kubernetes API at all — attestation is purely TPM-driven. The Role (managing the spire-bundle ConfigMap) was kept.

Phase 5 — SPIRE Agent Config and DaemonSet (complete)

manifests/spire/agent-config.yaml: k8s_psat replaced with tpm_devid:

```
NodeAttestor "tpm_devid" {
  plugin_data {
    devid_cert_path = "/etc/spire/devid.crt"
    devid_priv_path = "/etc/spire/devid.priv"
    devid_pub_path  = "/etc/spire/devid.pub"
    tpm_device_path = "/dev/tpmrm0"
  }
}
```

tpm_device_path is set explicitly. Without it, the plugin auto-detects, which failed inside the container even after the device was mounted.

manifests/spire/agent-daemonset.yaml: two volume changes: - **Added:** /dev/tpmrm0 as a CharDevice hostPath — the SPIRE agent must open the TPM device directly to execute the Load command and credential activation challenge. /dev/tpmrm0 (the resource manager) is used rather than /dev/tpm0 (the raw device) because the resource manager handles concurrent access from multiple processes. - **Removed:** projected service account token volume (spire-token) — no longer needed since node attestation is TPM-based, not PSAT-based.

Phase 6 — Deploy and Re-register (complete, 2026-07-09)

Server rolled first. Log confirms:

Plugin loaded: plugin_name=tpm_devid plugin_type=NodeAttestor

All 6 agents attested successfully. Each node received a unique SPIFFE ID derived from the SHA-1 fingerprint of its DevID certificate:

Node	SPIFFE ID (agent)
ipc4	spiffe://ipc.local/spire/agent/tpm_devid/25a1f67bfc2a6dc830de61d8dd2b5e78
ipc5	spiffe://ipc.local/spire/agent/tpm_devid/37bec218b1f39af8e7db42f04d710a3a
ipc6	spiffe://ipc.local/spire/agent/tpm_devid/ab96c7507b860042e3711446552e8d2e
ipc7	spiffe://ipc.local/spire/agent/tpm_devid/8b84df1549516ddba24415505b989fbb
ipc8	spiffe://ipc.local/spire/agent/tpm_devid/27bd47f544c71bd8ed20ec07c546a225
ipc9	spiffe://ipc.local/spire/agent/tpm_devid/dd04bfbf9dbfb604b0fdc1bf1d42d03e

Registration entries updated: a new node alias entry was added alongside the existing k8s_psat entry:

SPIFFE ID: spiffe://ipc.local/k8s-node

Parent: spiffe://ipc.local/spire/server

Selector: tpm_devid:issuer:cn:ipc-cluster DevID CA

-node flag set

The issuer:cn selector matches all nodes signed by our DevID CA without needing per-node entries. Workload entries (demo-app, mtls-server, etc.) were unchanged — they reference spiffe://ipc.local/k8s-node as their parent, which the new alias entry satisfies.

End-to-end verification: the spire-demo/demo-workload pod received SVID spiffe://ipc.local/demo-app (1-hour TTL, signed by the SPIRE CA).

Phase 7 — Reinstall Automation (complete)

scripts/reinstall-nodes.sh calls provision-tpm-devid.sh after a node rejoins k3s and is Ready. If op is not authenticated, the step warns but does not fail the reinstall. The operator can re-run bash scripts/provision-tpm-devid.sh <node> manually afterward.

Operational loop for a reinstalled node: 1. PXE reinstall completes 2. Node rejoins k3s (join-server.sh or upgrade-agents.sh) 3. provision-tpm-devid.sh generates a new TPM-bound DevID key, fetches CA key from 1Password, signs a new cert, installs at /etc/spire/ 4. Flux reconciles SPIRE agent DaemonSet — agent starts, loads DevID blobs, attests via TPM 5. SPIRE server issues Node SVID matching tpm_devid:issuer:cn:ipc-cluster DevID CA 6. Workloads on the node can receive SVIDs

No manual SPIRE steps are needed after a node reinstall.

TPM Server Attestation

The `spiffe/spire-server-attestor-tpm` project (v0.0.4, April 2025) closes this gap. It provides four binaries that together give agents a cryptographic way to verify the server before accepting its bundle.

Components

Binary	Role	Runs on
<code>spire-server-attestor-tpm-sign</code>	SPIRE <code>BundlePublisher</code> plugin; receives the trust bundle from the server and pushes it to the signer	SPIRE server pod (hostPath binary)
<code>spire-server-attestor-tpm-signer-unix</code>	Root daemon with TPM access; signs the bundle using the server's TPM key; writes signed JWT to disk	SPIRE server pod (hostPath binary, <code>/dev/tpmrm0</code> mounted)
<code>spire-server-attestor-tpm-signer-http</code>	Not used on this cluster. Despite its name, this is a software-key signing alternative to <code>signer-unix</code> — it signs JWTs using a PEM RSA private key, not the TPM. It is not an HTTP frontend for <code>signer-unix</code> . Using it here would replace hardware attestation with a software key.	—
<code>spire-server-attestor-tpm-verifier</code>	Not used on this cluster. Agent-side daemon intended for systemd deployments; replaced here by the <code>verify-bundle</code> init container.	—

The project ships all four binaries. Two are appropriate for this cluster's Kubernetes/TPM deployment; two are not:

- **sign + signer-unix**: use these. They provide the TPM-backed signing chain.

- **signer-http**: skip. It is for deployments *without* a TPM, where a software key is acceptable. Deploying it alongside **signer-unix** would serve a separately signed (software-keyed) bundle that agents could not verify against the TPM public key anyway.
- **verifier**: skip. Designed as a persistent daemon for systemd-managed agents. Replaced here by the **verify-bundle** init container, which is better suited to the Kubernetes pod lifecycle (runs once, exits, agent starts).

The HTTP serving role that **signer-http** would fill in a non-TPM deployment is handled here by a busybox **httpd** sidecar — one line, no config, serves the file that **signer-unix** writes.

Data Flow

Why This Breaks the Circular Trust

The verifier must verify the HTTP bundle before the SPIRE agent accepts it. The verification requires the server's TPM public key. That public key is NOT fetched from Kubernetes — it is written to `/etc/spire/server-bundle-signing.pub` on each agent node during physical provisioning (by `scripts/provision-tpm-devid.sh`), before any SPIRE component starts.

This is the out-of-band anchor. An attacker who compromises the Kubernetes control plane and rewrites the **spire-bundle** ConfigMap or the **signer-http** response still cannot produce a valid signature over the substituted bundle because:

- Producing the signature requires the server's TPM private key
- That private key never leaves ipc4's TPM chip
- The verifier checks the signature before the SPIRE agent accepts anything
- Signature check fails → agent rejects the bundle → does not connect to the rogue server

The server's TPM public key on the agent nodes is the cryptographic root of the entire bootstrap chain. It is distributed through the same physical provisioning channel as the agent's own DevID material, not through Kubernetes.

Server TPM Key (separate from DevID)

A dedicated **bundle-signing key** is provisioned on ipc4's TPM, separate from the agent's DevID key. Key separation is important: the DevID key is used for SPIRE agent attestation; the bundle-signing key is used only to authenticate the trust bundle to agents. Different purposes, different keys, both TPM-bound.

The bundle-signing key uses TPM handle `0x81008006` (the handle recommended by the project). It is created under the Owner hierarchy (same as DevID keys) with RSA 2048 and **rsassa** signing scheme.

Integration for This Cluster

Because our SPIRE server and agents run as Kubernetes pods rather than systemd services, the components are adapted as pod sidecars:

- **signer-unix** runs as a sidecar container in the **spire-server** StatefulSet pod with `/dev/tpmrm0` mounted. The **sign** and **signer-unix** binaries are hostPath volumes from `/usr/local/bin/` on ipc4. **signer-http** is NOT used — see the Components table above.

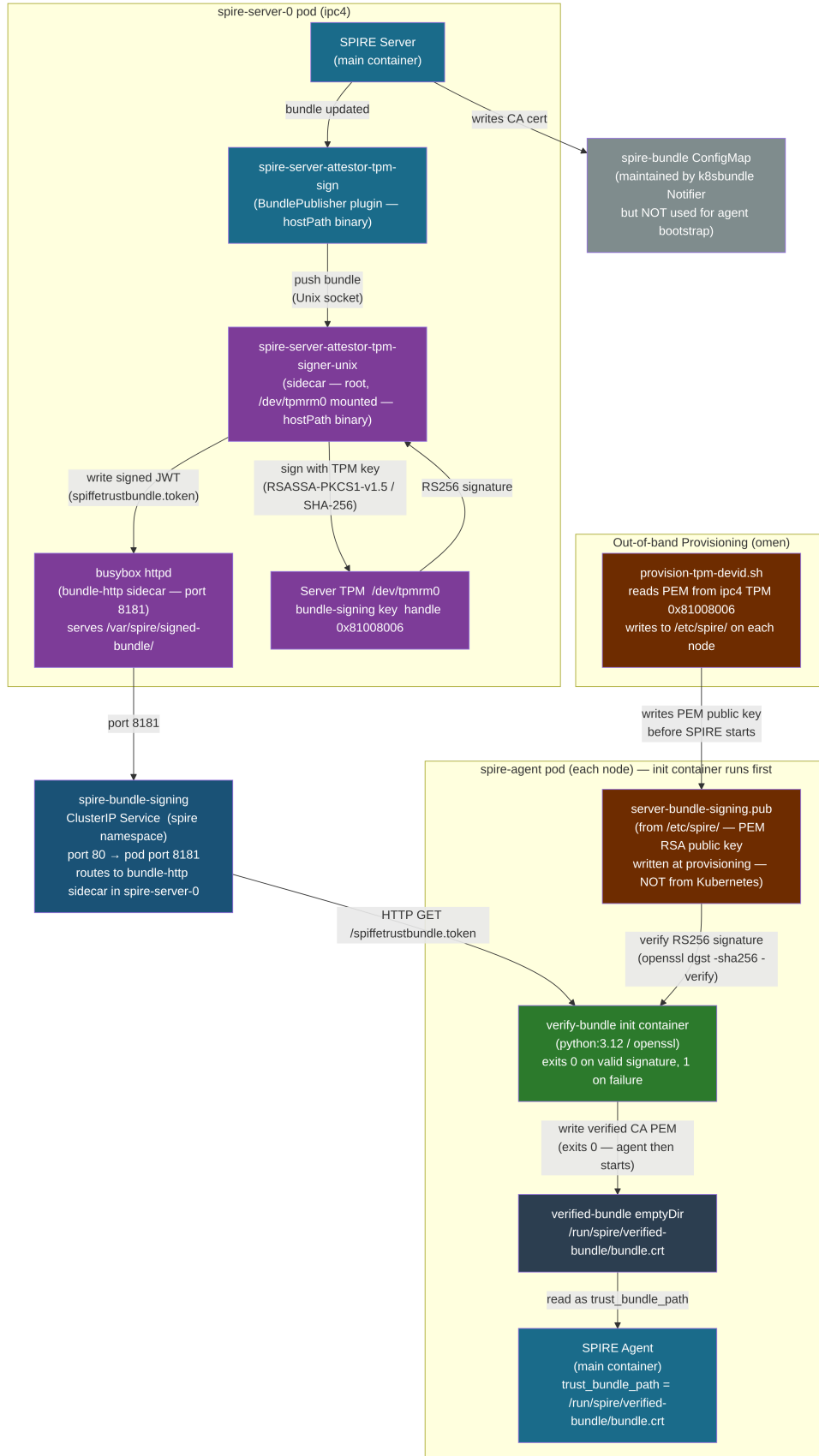


Figure 4: TPM server bundle signing and verification flow

- `signer-unix` writes the signed JWT to `/var/spire/signed-bundle/spiffetrustbundle.token` on a shared emptyDir. A `bundle-http` sidecar (`busybox:1.36`) serves that directory over HTTP with `httpd -f -p 8181 -h /var/spire/signed-bundle` — no config file needed. This is exposed as the `spire-bundle-signing` ClusterIP Service on port 80.
- Instead of the `verifier` binary, a `verify-bundle init container` (`python:3.12`) runs at agent pod startup. It fetches the signed JWT from the `spire-bundle-signing` ClusterIP Service, verifies the RS256 signature using `openssl dgst -sha256 -verify` against `/etc/spire/server-bundle-signing.pub`, extracts the `spiffetb` payload, and writes it to `/run/spire/verified-bundle/bundle.crt` on an emptyDir volume shared with the main container. The agent pod does not start until this init container exits 0.
- The SPIRE agent uses `trust_bundle_path = "/run/spire/verified-bundle/bundle.crt"` (the emptyDir written by the init container) instead of the `spire-bundle` ConfigMap.
- `scripts/provision-tpm-devid.sh` also writes `/etc/spire/server-bundle-signing.pub` on every agent node.

Resulting Security Posture

Direction	Before (k8s_psat era)	After tpm_devid	After server attestation
Server trusts agent	Kubernetes TokenReview (policy)	TPM credential activation (hardware)	TPM credential activation (hardware)
Agent trusts server	ConfigMap (unauthenticated)	ConfigMap (unauthenticated)	TPM signature on bundle (hardware)
Attacker must compromise	RBAC policy	Physical TPM on agent node	Physical TPM on both ipc4 AND agent node
Can agent detect rogue server?	No	No	Yes — signature check fails

Experiments Running on This Cluster

Experiment 11: Workload SVID Fetch

Namespace `spire-demo`, service account `demo-sa`. A demo pod fetches its X.509 SVID via the Workload API and prints the SPIFFE ID and cert details using `openssl`. Proves end-to-end attestation works.

Registration entries created by an imperative Job:

SPIFFE ID	Parent	Selectors
<code>spiffe://ipc.local/k8s-node</code>	<code>spiffe://ipc.local/spire/tpm_devid</code>	<code>tpm_devid:issuer:cn:ipc-cluster DevID CA (active node alias)</code>
<code>spiffe://ipc.local/k8s-node</code>	<code>spiffe://ipc.local/spire/k8s_server</code>	<code>k8s_server:cluster:ipc (legacy — exists in DB, unused)</code>

SPIFFE ID	Parent	Selectors
<code>spiffe://ipc.local/demo-app</code>	<code>spiffe://ipc.local/k8s-k8s:ns:spire-demo,</code> <code>node</code>	<code>k8s:sa:demo-sa</code>

Experiment 21: mTLS Between Two Workloads

Namespace `mtls-demo`. An `mtls-server` and `mtls-client` each receive unique X.509 SVIDs from SPIRE. The server runs `openssl s_server` requiring client cert (`-Verify 1`); the client runs `openssl s_client` presenting its SVID. Both sides verify against the SPIRE trust bundle. Zero pre-shared secrets.

SPIFFE ID	Workload
<code>spiffe://ipc.local/mtls-server</code>	<code>mtls-server</code> Deployment (<code>mtls-server-sa</code>)
<code>spiffe://ipc.local/mtls-client</code>	<code>mtls-client</code> Pod (<code>mtls-client-sa</code>)

Advanced Topics and Production Considerations

SVID Rotation for Long-Running Workloads

The experiments use an **init-container pattern**: fetch SVID once at startup, write to an emptyDir, main container uses it. This is fine for demos but won't handle rotation — after ~1 hour the cert expires and the workload has stale credentials.

For production workloads, the options are:

- **spiffe-helper**: a sidecar that watches the Workload API socket for updated SVIDs and rewrites cert files on disk, then sends `SIGHUP` to the main process.
- **SPIFFE SDK** (Go, Java, Python): native library that holds a live connection to the Workload API and delivers updated SVIDs to the application via callback.
- **Envoy SDS** (Secret Discovery Service): Envoy natively speaks the SPIFFE Workload API; it handles rotation transparently without application changes.

SPIRE HA

The current server is a single StatefulSet replica with SQLite. SQLite cannot be shared across replicas. High availability requires:

1. Switch the datastore to PostgreSQL or MySQL
2. Run multiple server replicas behind a load balancer

With a single server, a server restart means agents can't renew SVIDs during the outage. Workloads continue with cached SVIDs until they expire. Acceptable for a homelab; a consideration for production.

Federation

SPIRE supports federating trust between different SPIFFE trust domains. For example, `ipc.local` and `cloud.example.com` could exchange trust bundles so workloads in each cluster can verify SVIDs

from the other. Not needed until there are multiple clusters or external partners, but the mechanism is built in to SPIRE.

The Trust Root and Security Boundary

With `tpm_devid`, the server’s trust in the agent is hardware-rooted: the agent proves possession of a TPM-bound key and that the key and EK live in the same physical chip. This is strong, asymmetric cryptographic proof.

The agent’s trust in the server is not symmetric. It is Kubernetes-rooted.

The bootstrap problem: before the agent can attest, it needs to establish a TLS connection to the SPIRE server. To verify the server’s TLS certificate it needs a trust anchor — the SPIRE CA cert — which arrives in the `spire-bundle` ConfigMap. But the ConfigMap is not itself cryptographically authenticated. The agent has no independent way to verify “this CA cert belongs to *the* SPIRE server I am supposed to talk to.”

What the agent actually knows at bootstrap:

1. There is a CA cert at `/run/spire/bundle/bundle.crt`
2. The server it can reach presents a TLS cert signed by that CA
3. Those two facts are mutually consistent

Consistency is not proof of legitimacy. A rogue SPIRE server could publish its own CA cert into the ConfigMap and present a cert signed by it. From the agent’s perspective, both situations look identical — cert matches CA, TLS handshake succeeds, proceed to attest. The agent cannot detect the substitution.

The circularity: the agent uses the bundle to authenticate the server, but the bundle itself arrived from an unauthenticated channel. Breaking this circularity requires an out-of-band trust anchor — something the agent knows before it ever contacts the server.

Current posture: the ConfigMap write is protected by RBAC. Only the `spire-server` ServiceAccount can write `spire-bundle`. This is a policy control, not a cryptographic proof. If that policy is ever violated — through a compromised ServiceAccount, a misconfigured RBAC rule, or a cluster-admin mistake — the agent cannot detect it. The `spire` namespace is the security boundary, not individual workloads.

Vault Integration via JWT SVID

A natural next step for this cluster (Vault is already deployed): pods can authenticate to Vault using a SPIRE-issued JWT SVID via Vault’s JWT auth method. No Vault tokens or secrets need to be injected into pods. The flow is:

1. Pod fetches JWT SVID (audience: `vault`)
2. Pod calls `vault write auth/jwt/login jwt=<svid> role=<role>`
3. Vault validates the JWT against the SPIRE trust bundle (configured as a JWKS endpoint)
4. Vault returns a short-lived Vault token

This replaces Vault’s Kubernetes auth method with a stronger, SPIFFE-native identity.

Failure, Recovery, and Debugging

What Breaks When the SPIRE Server Goes Down

SPIRE agents cache SVIDs in memory. A server outage does not cause immediate failure — the cascade is time-dependent:

T+0 (server dies) - Running workloads with cached SVIDs: **unaffected**. The agent serves the cached credential and the workload keeps working normally. - New workload pods starting now: **blocked**. The agent cannot reach the server to sign a new SVID. The pod starts but any call to the Workload API returns an error. - New node joins / agent restarts: **blocked**. Node attestation requires the server. An agent that hasn't yet attested cannot operate at all.

T+30 min (50% of 1-hour TTL) - The agent starts attempting SVID renewal in the background. Renewal fails silently. The workload is still using the cached SVID and sees nothing wrong yet.

T+1 hr (SVIDs expire) - Cached SVIDs are now invalid. What breaks depends on how the SVID is used:

mTLS between services — the TLS handshake fails because the presented certificate is expired. The connecting service's TLS library rejects it with a certificate expiry error. If the service is `mtls-client` calling `mtls-server`, the call fails with a TLS error. If the service retries, it keeps failing until the server comes back and the agent can issue a fresh SVID.

Vault authentication via JWT SVID — the JWT has an `exp` claim. Vault validates it on every login call and rejects expired tokens with `token is expired`. A pod that fetches a Vault token at startup and holds it may continue working (Vault tokens have their own TTL), but any pod that re-authenticates to Vault (e.g. to rotate secrets) will fail until a fresh JWT SVID is available.

Envoy mTLS sidecar — Envoy holds the SVID and rotates it via the SPIFFE Workload API. When the SVID expires and rotation fails, Envoy's outbound TLS connections start failing with certificate errors. Inbound connections from peers will also be rejected if the peer's Envoy enforces certificate validity.

X.509 SVID used for signing or encryption — any operation that presents the certificate to a third party (another service, an HSM, a PKI system) fails because the peer rejects the expired cert.

The common thread: **the failure appears at the peer, not at the workload itself**. The workload makes what looks like a normal network call; the remote end rejects it because the credential is stale. This can look like a network problem or a service outage rather than an identity problem if you don't know to look at certificate expiry.

Recovery — once the SPIRE server is back, agents reconnect, re-attest if needed, and renew SVIDs automatically. Workloads that hold a connection open get the new SVID pushed (if using the SPIFFE SDK or Envoy SDS). Workloads using a one-shot init-container pattern need their pod restarted to pick up the new credential.

For this cluster the server is a StatefulSet with NFS-backed storage. If the pod is killed, Kubernetes reschedules it automatically — typically within a minute, well inside the 1-hour SVID TTL. The realistic risk is not a sustained server outage but a transient restart during which no new SVIDs can be issued.

Debugging SPIRE Problems

Check server health:

```
kubectl exec -n spire spire-server-0 -c spire-server -- \
  /opt/spire/bin/spire-server healthcheck
```

List attested agents:

```
kubectl exec -n spire spire-server-0 -c spire-server -- \
  /opt/spire/bin/spire-server agent list
```

Each entry shows the attestation type, SPIFFE ID, expiry, and whether it can re-attest. If a node is missing, its agent never attested — check agent logs and TPM provisioning.

List registration entries:

```
kubectl exec -n spire spire-server-0 -c spire-server -- \
  /opt/spire/bin/spire-server entry show
```

Check agent logs for workload attestation failures:

```
kubectl logs -n spire <agent-pod> -c spire-agent | grep -E 'error|SVID|attest'
```

Common errors:

- No identity issued — the workload connected but no registration entry matched its namespace/service account. Check `entry show` and verify the pod's SA.
- failed to attest — node attestation failed. Check TPM device access and DevID cert validity (`sudo openssl x509 -in /etc/spire/devid.crt -noout -dates`).
- could not verify bundle signature — the `verify-bundle` init container failed. Check that `/etc/spire/server-bundle-signing.pub` on the node is a valid PEM public key (`sudo openssl pkey -in /etc/spire/server-bundle-signing.pub -pubin -noout`).

Check the signed bundle is being produced:

```
kubectl exec -n spire spire-server-0 -c signer-unix -- \
  ls -la /var/spire/signed-bundle/
```

If `spiffetrustbundle.token` is absent, `signer-unix` has not yet received a bundle push from the `tpm-sign BundlePublisher`. It runs every 5 minutes — wait and check again.

Verify the HTTP service is reachable from an agent node:

```
kubectl exec -n spire <agent-pod> -c spire-agent -- \
  wget -qO- http://spire-bundle-signing.spire.svc.cluster.local/spiffetrustbundle.token \
  | head -c 80
```

Force SVID fetch from a running agent (expects a registered workload):

```
kubectl exec -n spire <agent-pod> -c spire-agent -- \
  /opt/spire/bin/spire-agent api fetch x509 -socketPath /run/spire/sockets/agent.sock
```

Returns `PermissionDenied: no identity issued for unregistered callers (the agent container itself)` — this is correct and means the agent is healthy.

Configuration Reference

Key Files

Path	Purpose
manifests/spire/server-config.yaml	SPIRE Server ConfigMap (trust domain, attestors, storage)
manifests/spire/agent-config.yaml	SPIRE Agent ConfigMap (server address, attestors, socket path)
manifests/spire/server-statefulset.yaml	Server StatefulSet + headless service
manifests/spire/agent-daemonset.yaml	Agent DaemonSet (hostPID, tolerations, socket hostPath)
manifests/spire/server-rbac.yaml	Server SA, ClusterRole (ConfigMap write for spire-bundle only; TokenReview/nodes/pods removed when switching to tpm_devid)
manifests/spire/agent-rbac.yaml	Agent SA, ClusterRole (pods, nodes, nodes/proxy)
clusters/ipc/spire.yaml	Flux Kustomization (path: ./manifests/spire, interval: 10m)
experiments/11-spire/	Workload SVID fetch demo
experiments/21-mtls-spire/	mTLS between two workloads

Server Configuration Summary

Setting	Value
Trust domain	ipc.local
Node attestor	tpm_devid; DevID CA: config/tpm/devid-ca.pem; EK CA bundle: config/tpm/endorsement-ca-bundle.pem
Key manager	Disk (/run/spire/data/keys.json)
Datastore	SQLite3 (/run/spire/data/datastore.sqlite3)
Notifier	k8s_bundle → ConfigMap spire-bundle
BundlePublisher	tpm-sign plugin → signer-unix socket; frequency 5m
gRPC port	8081
Health port	8080 (/live, /ready)

Agent Configuration Summary

Setting	Value
Trust domain	ipc.local
Server	spire-server:8081

Setting	Value
Trust bundle	<code>/run/spire/verified-bundle/bundle.crt</code> (emptyDir written by <code>verify-bundle</code> init container)
Workload API socket	<code>/run/spire/sockets/agent.sock</code>
Node attestor	<code>tpm_devid</code> ; DevID blobs from <code>/etc/spire/</code> ; <code>/dev/tpmrm0</code>
Workload attestor	<code>k8s</code> , <code>skip_kubelet_verification: true</code> , <code>use_new_container_locator: true</code>
Key manager	Memory
Health port	8080

Appendix: Design Notes

Trust Bundle Bootstrap — Standard Options and What We Built

Standard SPIRE trust bundle bootstrap options:

SPIRE provides four built-in ways for an agent to obtain the initial trust bundle at startup:

Option	Mechanism	Security property
<code>spire-bundle</code> ConfigMap (<code>k8s_bundle</code> notifier)	Server writes bundle to ConfigMap; agent mounts it	Kubernetes RBAC — policy, not cryptographic proof
<code>trust_bundle_path</code>	PEM file pre-placed on disk	Filesystem permissions — integrity depends on delivery channel
<code>trust_bundle_url</code>	Fetch bundle from an HTTPS URL at startup	HTTPS certificate — pushes trust question to whoever runs the URL
<code>insecure_bootstrap</code>	Accept whatever the server presents	None — only appropriate on fully-trusted private networks

All four have the same fundamental limitation: the bundle arrives through a channel that is either Kubernetes-controlled or unauthenticated. A compromised Kubernetes control plane can substitute the ConfigMap; a `trust_bundle_url` endpoint can be hijacked; a pre-placed file can be replaced if the node is compromised before SPIRE starts.

What we built:

Our implementation is functionally equivalent to `trust_bundle_url` combined with `spire-server-attestor-tpm` signing: the bundle is fetched from an HTTP endpoint (`spire-bundle-signing` ClusterIP Service), but the content is a JWT signed by the server's TPM private key (handle `0x81008006` on `ipc4`). The agent node has the matching public key installed out-of-band via `provision-tpm-devid.sh` before SPIRE starts, through a channel that Kubernetes cannot influence.

Rather than using SPIRE's native `trust_bundle_url` config option (which would require the SPIRE agent binary to perform the signature verification), we use `trust_bundle_path` pointing at an `emptyDir` volume, and a `verify-bundle` init container performs the fetch-and-verify step before the agent starts. This fits the Kubernetes pod lifecycle cleanly — the init container runs once, exits 0 on success, and the agent reads the verified bundle from the shared `emptyDir`. The `spire-server-attestor-tpm` project's `verifier` binary was designed for the same role but as a persistent `systemd` daemon, which does not map well to Kubernetes pods (and Pelagos does not support native sidecars with `restartPolicy: Always`).

The net result is a bootstrap path whose integrity guarantee is hardware-rooted: producing a valid signature over a substituted bundle requires physical access to ipc4's TPM.